

QA Report — Full API Functional & Security Test Pass

Date: 2026-09-21 **Branch tested:** `production` (merge of PR #29 / `devG`) **Build:** Spring Boot + PostgreSQL (prod), H2 `MODE=PostgreSQL` (test profile) **Result: 416 tests green** — 360 pre-existing + **56 new QA tests** written for this pass **Suite added:** `src/test/java/com/lawfirm/erp/qa/`

SuperAdminQaTest	16 tests	Super Admin end-to-end
FirmRoleMatrixQaTest	10 tests	every role, cross-role access, custom role
CaseManagementQaTest	9 tests	case work per firm + isolation
ProjectManagementQaTest	8 tests	projects, client portal, credentials, renewals
AuthSecurityQaTest	13 tests	auth, recovery, tokens, hardening

Run them with:

```
./mvnw -o test -
Dtest='SuperAdminQaTest,FirmRoleMatrixQaTest,CaseManagementQaTest,ProjectManagementQaTest,AuthS
```

1. Method — what "tested" means here

Every test drives the **real HTTP layer** through `MockMvc` against a full Spring context, so the JWT filter, `@PreAuthorize` , `PermissionEvaluator` , firm-context holder, service layer, JPA and the audit/notification side-effects all execute. Tokens are minted the same way the login flow mints them (including the `permVersion` claim), so the permission-staleness rules are exercised for real.

- 229 endpoint mappings across 38 controllers were inventoried and grouped by module.
- Each flow was tested as the *user* it belongs to: Super Admin → firm admin → each firm role → client, plus anonymous/other-firm callers for the negative cases.
- Where a behaviour is a known gap rather than a passing test, the test **asserts the current behaviour and says FINDING / GAP in the message**, so the finding is reproducible instead of being a permanently red build.

API surface tested (inventory)

Area	Endpoints	Notes
Auth	7	login, client login, refresh, MFA setup/validate, change-password
Super Admin	31	firms, firm admins, modules, roles, config, audit, users, invoices
RBAC (platform)	22	roles, permissions, modules, tenant types
Firm management	41	profile, config, email config, employees, clients, roles, modules, audit
Case management	33	matters, court cases, hearings, assignments, calendar, dashboard
Project management	25	projects, members, credentials, renewals, renewal types, client portal
User management	9	list, search, profile, permissions, activity, reset password/MFA, bulk
Notification	7	list, unread count, read, broadcast, preferences
Scraper / hearings	8	courts, scrape, export, hearing status, matches

Me / dashboards	3	profile, case dashboard, global dashboard
-----------------	---	---

2. Super Admin — every function exercised

#	Function	Result
SA-01	Register with a wrong secret	✓ rejected 400
SA-02	Login always challenges for MFA	✓ MFA_SETUP_REQUIRED + mfaToken, never a naked access token
SA-03	Create firm → admin created, 4 roles cloned, admin forced to change password on first login	✓ data correct
SA-04	List all firms (GET /super-admin/firms)	✗ endpoint missing (see F-4)
SA-05	Grant <i>all</i> permissions to the firm admin	✓ admin then reaches matters, users, clients, employees
SA-06	Grant a <i>narrow</i> set to another firm	✓ narrow firm can read matters, blocked from create + users; wide firm unaffected (no permission bleed)
SA-07	Firm-admin listing, by firm, by id	✓ data matches created admin
SA-08	Suspend / activate firm	⚠ status flips correctly, but the suspended firm's users keep working (F-1)
SA-09	Trial create → extend → convert to permanent	✓ expiry moves forward, isTrial cleared, status ACTIVE
SA-10	Module enablement per firm	✓ nothing enabled at creation, enable works, status reported
SA-11	Disable a module → does it block the API?	✗ no (F-6)
SA-12	SA resets a firm admin's password	✓ new password authenticates
SA-13	SA resets MFA	✓ secret cleared, flag kept (by design), fresh QR issued on next login
SA-14	SA creates a custom firm role	⚠ role created, but permissions sent on create are dropped (F-7)
SA-15	Firm admin vs Super Admin endpoints	✓ 403 on SA users/firms/audit/roles/tenant-types
SA-16	SA audit trail	✓ responds (native date() SQL is Postgres-only, see note in §8)

Dependency chain found: a freshly created firm is *inert* — its `FIRM_ADMIN` role has **zero permissions** and **no modules are enabled**. Until the Super Admin performs two manual steps (grant the ceiling + enable modules) the firm admin cannot do anything, and if modules were ever enforced they could not log in usefully at all. The onboarding flow has no "recommended defaults".

3. Roles inside a firm — individually tested

Setup for these: SA grants the firm's `FIRM_ADMIN` everything, then the firm admin assigns each role its own permission set — exactly how the product is meant to be operated.

#	Scenario	Result
ROLE-01	Firm admin full toolkit: roles CRUD, employee CRUD + role change + toggle, client CRUD + portal toggle, user management (profile/permissions/activity/reset password/reset MFA), audit, dashboards	✓
ROLE-02	ADVOCATE: creates and reads matters, calendar; denied roles, employees, clients, all-users, firm config, email config, audit, platform roles, broadcast	✓ 403 on all
ROLE-03	PARALEGAL read-only: reads matters, denied create/edit matter, roles, employees	✓
ROLE-04	Privilege escalation: advocate holding <code>ROLE_MANAGEMENT:VIEW/EDIT</code> tries to grant itself every permission	✓ 403, role row unchanged (0 rows written)
ROLE-05	Firm admin tries to grant a permission above its own ceiling	✓ 403, target role stays empty
ROLE-06	Custom role (<code>CASE_READER</code> , exactly <code>VIEW</code>) assigned to a user	✓ can view, denied create/clients/users/dashboard
ROLE-07	Downgrade employee <code>ADVOCATE</code> → <code>PARALEGAL</code>	✓ old token refused, new token read-only (see F-10 for the UX trap)
ROLE-08	Delete cloned default role vs delete custom role	✓ default blocked with a clear message, custom deleted
ROLE-09	<code>CLIENT</code> role granted case view	✗ sees the whole firm's matters (F-3)
ROLE-10	<code>GET /modules/users/{id}/activity</code> without <code>USER_MANAGEMENT</code>	✗ allowed (F-5)

Answer to "can one role view/edit another role?" — No. Nothing in a role's permission set lets it modify another role: `FirmRoleController` is `@PreAuthorize("hasRole('FIRM_ADMIN')")`, and a non-`FIRM_ADMIN` role gets 403 even when it holds `ROLE_MANAGEMENT:*`. Roles also cannot read each other's *data* beyond the firm-admin-only role endpoints, and firm A cannot touch firm B's roles.

Custom-role limitation: custom roles work for permission-gated APIs (`CASE_MANAGEMENT:*`, `PROJECT_MANAGEMENT:*`, ...) but **not** for the endpoints guarded by `hasRole('FIRM_ADMIN')` — employees,

clients, firm roles, firm config/email config. A custom role can be given `EMPLOYEE:CREATE` and still get 403. That surprises users who expect permissions to be the only switch.

4. Case Management — per firm, with lawyers/employees

#	Scenario	Result
CASE-01	Matter with a client party → auto-filed court case → second court case → hearing → timeline → stale list → search → calendar → appeal deadlines → dashboards	✓ data correct & linked
CASE-02	Legal workflow: illegal stage jump blocked; walked FILED → ... → JUDGMENT_AWAITED via <code>allowed-stages</code> ; judgment recorded → JUDGMENT_DELIVERED	✓
CASE-03	Assign primary advocate + paralegal to a matter, list, revoke	✓
CASE-04	Firm B reads/edits/lists Firm A's matter	✓ blocked, Firm A data untouched
CASE-05	Firm B reaching Firm A's court case, hearings and stage endpoint	✓ blocked
CASE-06	Employee with no case permissions	✓ 403 on list/create/calendar/dashboard
CASE-07	Deactivated employee's existing token	✓ refused
CASE-08	Is a matter bound to a client?	✗ only at party level (F-3)
CASE-09	Anonymous / garbage token	✓ 401

Matters are not bound to clients. `Matter` has `firmId`, `matterNumber`, `matterType`, `title`, `status`, `currentCourtCaseId`, `assignedPartnerId`, `originatingCourtLevel`, `description` — and **no client reference at all**. `MatterResponse` likewise exposes no client. The only client link that exists today is `MatterParty.clientId` + `isOurClient`, and the hearing-reminder job reaches a client by looking up that party (recipient type `CLIENT`). Party matching (`POST /firm/matters/parties/match`) can find a client by name/mobile/e-mail, but there is no way to ask "which matters belong to this client?".

5. Project Management

#	Scenario	Result
PRJ-01	Project bound to client A → appears in A's portal only; unbound project never leaks into a portal; other client blocked by code	✓ scoping correct
PRJ-02	Bind a project to another firm's client	✓ rejected

PRJ-03	Update project, status change, members add/list/remove, dashboard, status filter	✓
PRJ-04	Firm B reads/edits/lists Firm A's project, injects a credential	✓ blocked
PRJ-05	Credential encrypted at rest; list gated by <code>CREDENTIAL_VIEW</code> ; reveal gated by <code>CREDENTIAL_REVEAL</code> ; update/delete	✓ cipher text never equals or contains the plain password
PRJ-06	Renewal type CRUD, renewal with YEARLY recurrence → instances generated, instance completed, renewal updated	✓
PRJ-07	Employee with no project permission on every endpoint	✓ 403
PRJ-08	Client portal vs firm-side project APIs	✓ blocked

Project management IS bound to clients (`Project.clientUserId` + denormalized `clientName` , indexed, with a client-scoped portal). So the two modules are inconsistent:

	Client link	Client portal	Own-records filtering
Case Management	✗ none (party only)	✗	✗
Project Management	✓ <code>clientUserId</code>	✓	✓ (portal filters by <code>clientUserId</code>)

Should Case Management be bound to clients? Yes — and here is the concrete case

Right now the permission matrix *tells* the product that a client has `CASE_MANAGEMENT: OWN` , and `DataInitializer` documents OWN as "only their own case". Nothing implements that, and nothing *can* implement it, because a matter has no client. The consequences with real data:

1. A client with case view sees every matter in the firm (confirmed: ROLE-09).
2. Hearing reminders to clients are best-effort via party matching — rename a party and the link breaks.
3. "My cases" cannot be built for the client portal without it.
4. Reports/handoffs can't answer "how many matters does this client have?".

Proposed shape (small, additive, matches the project module that already works):

```

matters.client_user_id UUID NULL → FK users(id)    (indexed, firm-scoped)
POST /api/v1/firm/matters { ..., clientUserId }    (optional, like projects)
GET  /api/v1/firm/matters?clientUserId=...         (filter)
GET  /api/v1/client/matters → client portal list, filtered by current user

```

Then implement OWN for real: when the caller's role scope is OWN, the repository query adds `client_user_id = currentUserId` (the same one-line pattern the project portal already uses). Party-level `isOurClient` can stay for co-parties/opposing counsel, but the *matter owner* should be a first-class column. Backfill is safe: leave existing rows NULL, and add the column as optional so nothing breaks for existing matters.

6. Forgot password / credential recovery

Path	Status
------	--------

Self-service "forgot password"	✗ does not exist — no route, no token, no mail. <code>grep -ri forgot</code> finds nothing in <code>src/main</code> OR docs.
User-initiated reset token by e-mail	✗ does not exist
Firm admin resets an employee's password	✓ works, forces rotation for a brand-new account
SA resets any password	✓ works, and immediately revokes the user's existing session (verified)
First login with a temporary password	✓ <code>PASSWORD_CHANGE_REQUIRED</code> → <code>change-password</code> → usable session; temp password dies afterwards
Change-password validation	✓ mismatch rejected, <8 chars rejected
Lockout	✓ 5 failed attempts → 30-minute lock, correct password also refused while locked
Enumeration	✓ unknown user / wrong password / unknown firm are indistinguishable 401s

Gaps in this area:

- **No self-service recovery.** If the only firm admin forgets their password, only a Super Admin can rescue them; if an employee forgets theirs, they must phone the firm admin. For a SaaS this is a support-load and a churn risk. This is the single biggest *missing feature* in auth.
- **Password policy is length-only and inconsistent:** `change-password` requires 8–50 chars, the admin/SA reset requires only 6–50. `"aaaaaa"` passes there. No complexity, no breach check, no reuse history.
- **Admin reset does not force rotation for an existing user** (only brand-new accounts carry `mustChangePassword`), so an admin-issued temporary password can be kept forever.
- **The temporary password is e-mailed in clear text** (`EmailService.sendPasswordReset(..., newPassword ...)`). Anyone with mailbox access owns the account.

7. Security assessment

Strong (verified by test)

Control	Evidence
Multi-tenant isolation on matters, court cases, hearings, projects, credentials, roles, users	CASE-04/05, PRJ-04, SA-15, ROLE-02, and the pre-existing <code>CrossFirmIsolationE2ETest</code>
Permission enforcement on every case/project endpoint (<code>CASE_MANAGEMENT:*</code> , <code>PROJECT_MANAGEMENT:*</code> , including <code>CREDENTIAL_VIEW/REVEAL</code>)	CASE-06, PRJ-05/07, ROLE-03
Super Admin role lock-out (<code>hasRole('SUPER_ADMIN')</code>)	SA-15
Permission changes invalidate live sessions (<code>permVersion</code>)	ROLE-07, and <code>simplifiedRbacE2ETest</code>
Password reset invalidates live sessions	AUTH-03

Limited-scope tokens (password-change, MFA) cannot call business APIs	AUTH-07
Tampered signature / payload / unsigned tokens rejected	AUTH-07
MFA mandatory for Super Admin, with QR re-enrolment after a reset	SA-02, SA-13
Brute-force lockout + no user enumeration	AUTH-04, AUTH-05
Secrets at rest: credential passwords encrypted, reveal separately permissioned	PRJ-05
Injection attempts fail closed, no 5xx, no SQL/stack-trace leakage	AUTH-10, AUTH-11
CORS is not wildcard	AUTH-13
Audit trail records privileged actions <i>and</i> failures	ROLE-01, audit log inspection

Weak / open (ranked)

#	Severity	Finding	Evidence
F-1	High	Suspended (or expired) firm's users still authenticate and use the API — <code>Firm.status</code> is never consulted at login or per-request	SA-08
F-2	High	Revoking client portal access does not stop the client logging in or using the portal — <code>User.portalAccessEnabled</code> is written but never read by auth	AUTH-09
F-3	High	Case Management has no matter→client binding and the documented own scope is never enforced → a client role sees the whole firm's matters	ROLE-09, CASE-08
F-4	High	<code>GET /api/v1/super-admin/firms</code> does not exist although <code>FirmService.getAllFirms()</code> + <code>FirmListResponse</code> exist and are unit-tested (the unused <code>FirmListResponse</code> import is the fingerprint of a lost edit)	SA-04
F-5	High	<code>GET /api/v1/modules/users/{userId}/activity</code> has its permission check commented out → any authenticated user can read any user's activity by UUID	ROLE-10
F-6	Medium	Module (plan) gating is dead code — <code>requireModuleAccess()/hasModuleAccess()</code> are never called, so disabling a module changes nothing but the UI	SA-11
F-7	Medium	<code>POST /roles</code> (both SA and firm admin) silently drops <code>permissionIds</code> → a "role with permissions" is created empty	SA-14, ROLE-06
F-8	Medium	Temporary password e-mailed in clear text; reset accepts 6-character passwords; no self-service recovery (see §6)	AUTH-01/02
F-9	Medium	No refresh-token revocation or rotation store — refresh is stateless, logout is client-side only, and a stolen refresh token is valid for its full 7-day life. Super Admin tokens also skip the <code>permVersion</code> staleness check	code review + AUTH-08

F-10	Medium	Role change requires re-login: the user's token is refused with 401 "Your permissions have changed" instead of being transparently downgraded. Correct for security, but the UI must handle it gracefully or users see unexplained logouts	ROLE-07
F-11	Low	@PreAuthorize is evaluated <i>after</i> @Valid, so unauthorized callers get 400 parameter errors instead of 403 — mild information disclosure about payload shape	ROLE-02
F-12	Low	New firm is inert: FIRM_ADMIN starts with zero permissions and zero modules (see §2)	SA-03/10
F-13	Low	Custom roles cannot use the hasRole(' FIRM_ADMIN ') endpoints even with the matching permissions (see §3)	ROLE-06
F-14	Low	Case/calendar/dashboard read endpoints require CASE_MANAGEMENT:VIEW; a firm admin granted everything is fine, but an employee without the module's VIEW cannot even see the calendar they're assigned to — consider a narrower CALENDAR:VIEW gate	ROLE-02, CASE-06

Notes that are *not* product bugs

- The global dashboard and audit list aggregate with a native `date(...)` query, which H2 (`MODE=PostgreSQL`) does not implement. They are therefore untested in this environment; they should be exercised against Postgres before release. Consider extracting "counts per day" into a derived query or a DB-agnostic grouping so the suite can cover them.
- Email delivery fails in the test profile (no From address) and is correctly recorded as an audit event rather than crashing the request.

8. Missing features inside existing features

Ordered by what a firm would hit first.

1. **Self-service password reset** (F-8) — no route, no token, no mail template.
2. **Matter → client binding + working "my cases"** (F-3) — blocks the client portal's case view.
3. **Firm list for the Super Admin console** (F-4) — the console cannot render firms today.
4. **Plan/module enforcement** (F-6) — the upsell mechanism is cosmetic; nothing stops a firm from using a module it was not sold.
5. **Suspension and portal-revocation actually taking effect** (F-1, F-2) — billing enforcement.
6. **Role with permissions in one call** (F-7) — create returns an empty role; the UI must do a second request, and if it forgets, the role silently does nothing.
7. **Logout / session revocation** (F-9) — no server-side endpoint to kill a session or a device.
8. **Document Management module is a stub** — 8 permissions seeded, no controller, no entity, no upload/download. Also `REPORTS` (4 permissions, no endpoints) and `BILLING` (7 permissions, no firm-facing endpoints; `InvoiceController` is platform billing at `/super-admin/invoices`, so `BILLING:*` gates nothing).
9. **Notifications are poll-only** — no WebSocket/SSE (no websocket starter in `pom.xml`), so in-app alerts lag until the next poll; no digest/e-mail-on-unread policy.
10. **No bulk user import**, no CSV export for matters/clients (reports module absent).
11. **No firm-admin "rescue me" path** for a lost MFA device other than the Super Admin.
12. **No audit-log export/retention policy** surfaced to firms.

13. **Employee create returns a generated username** (`firmcode` + compacted name) that differs from what the admin typed — the UI must display it, and no "username suggestion" is offered.
 14. **UserRole join table exists but is unused** — one role per user. "Advocate who is also the billing manager" is impossible without a second account.
 15. **API contract inconsistency**: most modules wrap bodies in `{"data": ...}`, project management takes bare DTOs, and `PATCH /projects/{code}/status` takes a query parameter. The frontend must special-case these; standardising (or documenting per endpoint) would prevent integration bugs.
-

9. Verdict

The multi-tenant core is in good shape: **isolation, permission gating, session invalidation on permission change, encrypted credentials, lockout, MFA and audit logging all held up** under adversarial tests (other-firm tokens, privilege-escalation attempts, injection payloads, forged JWTs). Nothing I threw at it produced a 5xx, a cross-tenant read, or an auth bypass on a permission-gated endpoint.

What is weak is everything *around* the core: **the product cannot cut off a suspended tenant, the client portal revocation switch does nothing, module/plan gating is cosmetic, one endpoint is missing its permission check, one is missing entirely, and password recovery has no self-service path**. Those five are worth fixing before the next customer onboards, because they are all "the switch looks like it works but does nothing" failures.

Recommended fix order: F-5 (2 lines) → F-2 (1 check) → F-1 (1 check) → F-4 (restore the mapping) → F-7 (use the permissionIds) → F-3 (matter client binding + OWN scope) → F-8 (recovery).

10. Companion documents

- `docs/ui-test-checklist.md` — what to test *from the UI*, module by module, so the product can be signed off end-to-end (the checklist to attach to the security audit).
- `docs/security-architecture-guide.md` — existing security architecture reference.
- `docs/rbac-roles-and-permissions.md`, `docs/ROLES_PERMISSION_V3.md` — RBAC reference.